

JBT Financial Midterm Report

David T. Wilkey
Developer
JBT Financial
Knoxville, TN

Bhumir Patel
Developer
JBT Financial
Knoxville, TN

Joey Syracuse
Developer
JBT Financial
Knoxville, TN

Abstract—Planning investments based on market trends and research is a time consuming process that many individuals are unwilling/unable to pursue. Even when extensive market research is performed, many investors are still left with a sense of uncertainty surrounding their financial choices. By leveraging linear regression, random forest, and k-nearest neighbor techniques, JBT Financial has developed machine learning tools which analyze vast amounts of financial data quickly and efficiently to provide the user with valuable insights to help guide their investment decisions. This paper provides an overview of the machine learning pipelines used throughout the development process and discusses the promising initial results established by baseline models.

I. INTRODUCTION

Investors have been interested in methods to best handle volatility and price fluctuations since the inception of the stock market. Despite performing hours of deep research and thoroughly evaluating company fundamentals, many traders are left dissatisfied with their investment performance. For others, lack of time or knowledge to devote to trading/market research is a significant barrier. The rise in popularity of mobile brokerage platforms has made stock trading more accessible for both large and small investors resulting in a greater need to address these unique challenges. JBT Financial has developed new machine learning tools designed to do exactly that.

Linear regression, random forest, and k-nearest neighbor machine learning techniques were utilized to create baseline models designed to provide the user with key pricing information to help guide their investment choices. To accomplish this, the models analyze vast amounts of historical trading data for a specific stock and then predict either the highest price point or closing price point of the stock at the end of the trading day. The following sections explore the design and performance of each model to provide a deeper understand of these tools and their application towards addressing important problems faced by today's stock traders.

II. LINEAR REGRESSION MODEL DEVELOPMENT

A. Data Exploration

Datasets used during training and testing of the baseline linear regression model were historical trading data for Berkshire Hathaway, Apple, and the Vanguard S&P 500 ETF. The baseline linear regression model was trained on the Berkshire Hathaway dataset. Basic data preparation tasks such as removing columns with missing values and converting dates

to datetime format were performed. A basic plot of the stock's price was created to visualize its change over time before training the model using the scikit-learn library. The feature matrix and target matrix were extracted from the dataset in a manner such that the features are used to predict the high price of the following day. A standard 80/20 train/test split was performed before fitting the model and evaluating the predictions made using the test set. Similar data cleaning was performed on the Apple and Vanguard ETF datasets. Minimal additional data preprocessing was also performed on the Vanguard dataset so it would match the formatting of the other datasets. Finally, the model was tested on unseen data.

B. Baseline Linear Regression Model

Testing of the baseline linear regression model was performed on the training data as well as two unseen datasets (Apple and Vanguard S&P 500 ETF). Prior to each testing run, basic plots of the stock price were visualized. After each testing run on unseen data, an additional plot was generated to compare the predicted model values with the actual values. An additional comparison table, mean squared error, and R2 score were also generated to further aid in performance evaluation.

Testing the model on data from the Berkshire Hathaway test set resulted in very high accuracy with an R2 score of 0.9991, suggesting the possibility of overfitting. To confirm this suspicion, the model was then tested on the unseen Apple dataset. As expected, the predicted prices were very inaccurate. For this dataset, a negative R2 score was calculated indicating the model explains none of the variability in the Apple data. An additional test run was then performed on the unseen Vanguard dataset. Due to the Vanguard dataset's reduced volatility in comparison with Apple, its target prices trend more closely to Berkshire Hathaway, suggesting better generalization to this dataset. Performance metrics for this test run confirmed this theory as evidenced by an R2 score of 0.9997 and mean squared error of 2.687.

In an attempt to increase the model's generalization to the Apple dataset, further data preprocessing was performed. Due to the much larger size of the Apple dataset, lower end outliers were manually trimmed from the data before re-evaluating performance. Unfortunately, this only yielded a marginal performance increase indicating the need for additional model refinements to counter overfitting.

Despite positive performance when applied to unseen data with many similarities to the training data, experimentation

with this baseline model highlights the limitations of basic linear regression. Possible improvements to the baseline linear model include introducing regularization, additional features, polynomial regression and/or increasing diversity of the training samples. Additional plans also include model modifications which output predictions for thirty days in the future instead of just the following day to increase practicality for real world use.

III. RANDOM FOREST MODEL DEVELOPMENT

A. Data Exploration

The datasets considered for this project consisted of 3 major investment types: a holding company, single stock, and an electronic trade fund (ETF). The specific investments evaluated were Berkshire Hathaway (BRK), Apple (AAPL), and Vanguard S&P 500 ETF (VOO). The intention in examining multiple types of investments was to explore how a machine learning approach might perform differently for different types of investments, as larger funds such as VOO and BRK exhibit less volatility than individual stocks such as AAPL, due to the diversification of their portfolios.

Each of the datasets for these three investments included basic metrics such as the open, close, low, and high prices for each trading day, as well as the volume traded during that day. The Apple dataset was by far the largest, starting from the company's IPO in 1980. This means that decades of lower prices are included in the training of the model which could influence the future predictions of the model. The VOO and BRK datasets only span from 2010 and 2015 to present respectively. Considering this more recent and condensed dataset and the fact that they are more diversified/stable investments, we would expect better performance on the predictions of these investments when compared to single-stock investments like AAPL.

B. Baseline Random Forest Model

For the purposes of this baseline model, the target prediction was the close price, while the other metrics were used as features (open, low, high, and volume). While random forests are most commonly used for classification problems, they can also be used for regression problems such as stock prediction with the scikit-learn RandomForestRegressor. This type of model works by creating an ensemble of decision trees to make predictions on a subset of samples from the overall dataset. Each tree is randomized and makes individual predictions about a single day's closing price, which are then averaged to make the final prediction. Ideally, ensemble methods such as random forests are used to control over-fitting, but significant evidence of overfitting was found in the use of this model. When training the random forests models a new model was trained on each dataset and then used for closing price prediction of that specific investment. The data was randomized into training and testing sets with the standard 80/20 split, as well as splitting the dates with the same randomization so the test data could later be resorted for plotting of the test set. For each investment, the RandomForestRegressor from

scikit-learn was trained and then evaluated on the test set. Each was trained with 500 estimators and a maximum tree depth of 10. A visualization was produced by sorting the dataframe containing the test set and dates to assess how close the predictions matched the test set visually, but the mean squared error and R2 score were also calculated to evaluate the performance. Each of the models' R2 scores were greater than 0.99, indicating that overfitting is likely to be occurring. The AAPL mean squared error was quite low, at 0.5871, while BRK and VOO had relatively similar mean squared errors of 10.16 and 9.891, which makes sense as they are in similar price ranges for the span of their datasets.

IV. K-NEAREST NEIGHBOR MODEL DEVELOPMENT

A. Data Exploration

The dataset utilized in this study consists of stock market prices obtained from the historical price data of Berkshire Hathaway Inc. The data includes several key attributes such as the opening price, highest price, lowest price, trading volume, and closing price. The dataset covers a significant time period, allowing the model to learn the inherent patterns in stock price movements.

Initially, a thorough exploration of the data was conducted to understand its structure and characteristics. The dataset contained multiple attributes, with the Close price being the target variable for prediction, while Open, High, Low, and Volume served as feature inputs. After parsing the date column and ensuring the data was chronologically ordered, missing or erroneous values were addressed to maintain data integrity. Furthermore, summary statistics such as mean, variance, and range were computed for each feature to identify any anomalies or outliers.

Feature scaling was a critical step due to the inherent differences in the range of the input variables, such as price values compared to trading volume. This ensured that the K-Nearest Neighbor algorithm, which relies on distance calculations between points, did not get biased toward features with larger numerical scales. As a result, the StandardScaler method was applied to normalize the features, standardizing them to a mean of 0 and a standard deviation of 1.

B. Baseline K-Nearest Neighbor Model

To establish a baseline for comparison, a K-Nearest Neighbor (KNN) regressor was implemented. KNN is a non-parametric algorithm that operates by identifying the 'k' most similar data points to a given input and using their values to predict the target variable. In this study, the number of neighbors (k) was initially set to 5, a common starting point in KNN implementations.

The feature set, consisting of Open, High, Low, and Volume, was divided into training and testing sets using an 80-20 split, ensuring that the temporal sequence of the data was preserved. This approach is crucial in time series forecasting as shuffling the data could disrupt the temporal dependencies. After splitting, the StandardScaler was applied to the training

set, and the same scaling transformation was applied to the testing set.

The KNN model was trained on the scaled training data, and predictions were made on the testing set. Performance evaluation was conducted using the Mean Squared Error (MSE) metric, which quantified the average squared difference between the predicted and actual closing prices. Initial results indicated that while KNN is capable of capturing some trends, the model's performance suffered from the lack of consideration for the sequential nature of the stock data, particularly in highly volatile markets. The MSE of the baseline model provided a benchmark for further refinement and comparison with other machine learning models.

To visualize the performance of the baseline model, the predicted stock prices were plotted against the actual closing prices over the test period. This revealed that the KNN model struggled to predict sudden price changes, which is consistent with the limitations of distance-based models in complex, non-linear time series data like stock prices.

V. PLANNED MODEL IMPROVEMENTS

There is much that can be done to improve these models as we move forward with development. We have determined several strategies that will serve to make these models more accurate and generalizable to work towards a model that will eventually be profitable in a real trading scenario. One of the clearest limitations we have found is the use of the KNN model, which we will phase out as we continue development. As an inherently simpler model than linear regression or random forests, it is not well suited to capturing the more intricate dynamics and broader trends that occur within market price movements. Another big change we are planning on making is in the way we perform the split for training and testing datasets. The traditional randomized split is not effective for time dependent data found in the stock market, so we will be moving to a sequential splitting method to preserve the time dependent nature of the dataset.

One strategy that we plan on using is the implementation of hyperparameter tuning for the linear regression and random forests models. This will be a useful tool in helping to find a balance for the models so that they will still be accurate while resisting overfitting, which has been a consistent issue across the baseline models we have implemented. Ultimately, this is a necessary addition to ensure that the models will provide realistic predictions, and will help us determine their accuracy more clearly.

We will also be expanding our features to include more relevant predictors, as we believe the use of just the raw data we have is likely to affect the performance of our models. By engineering features such as relative strength index (RSI), various ranges of moving averages, and moving average convergence divergence (MACD) and including them in the models, we believe they will have the potential to make more accurate predictions.

Overall, we believe these improvements as well as any others we implement along the way will serve to enhance

the model's predictive performance as well as its consistency in its predictions. We at JBT Financial hope to provide our customers with a product that will be reliable in its predictions, and a set of tools that will aid them in making more informed decisions about their investments.

VI. CONCLUSION

In conclusion, the development of machine learning tools at JBT Financial represents a significant stride towards empowering investors with insightful decision-making tools in the volatile landscape of financial markets. While initial models have shown promise, challenges such as overfitting and model generalization remain critical considerations.

By strategically discontinuing KNN and implementing planned improvements in hyperparameter tuning, feature engineering, and model diversification, JBT Financial aims to enhance the reliability and efficacy of our predictive models. These enhancements are pivotal in delivering robust tools that not only predict market trends accurately but also adapt to evolving market dynamics.

As we integrate these advancements and refine our methodologies, our commitment remains steadfast in providing investors with sophisticated tools that facilitate informed and strategic investment decisions. Through continuous innovation and adaptation, we aspire to set new benchmarks in the realm of financial forecasting and empower our clients to navigate markets with confidence and clarity.